

Operator: `MaxPool2d`

Phase 1.5 nấc 2 → đầu vào cho Phase 2A (viết operator) và Phase 2D (golden model C).

1. Vai trò

Chia ảnh thành các ô không chồng nhau, mỗi ô giữ lại **giá trị lớn nhất**. Giảm kích thước ngang-dọc, giữ nguyên số kênh.

Chia việc với `Conv2d` rất rõ:

	Đổi channel?	Đổi cao × rộng?	Có tham số?
<code>Conv2d(k=3, p=1)</code>	✓	✗	✓
<code>MaxPool2d(2, 2)</code>	✗	✓	✗ 0 params

2. Công thức

Cùng dạng với `Conv2d`, chỉ khác là mặc định `padding = 0`:

```
H_out = floor( (H_in - kernel) / stride ) + 1
```

Với `MaxPool2d(2, 2)` thì `kernel = stride = 2`, rút gọn thành `floor(H_in / 2)`.

Chỗ 7 → 3, không phải 3.5

```
floor( (7 - 2) / 2 ) + 1 = floor(2.5) + 1 = 2 + 1 = 3
```

`floor` áp lên **phép chia**, rồi mới `+1`. Ở đây hai cách viết ra cùng số vì cộng số nguyên giao hoán với `floor`, nhưng trong C phải viết đúng thứ tự.

Cái gì bị mất:

```
chỉ số:   0  1  2  3  4  5  6
cửa sổ 1 [0  1]
cửa sổ 2      [2  3]
cửa sổ 3          [4  5]
                        6 ← lè ra, KHÔNG đủ cặp → vút hoàn toàn
```

→ **Hàng cuối và cột cuối (chỉ số 6) bị bỏ hẳn**, không ảnh hưởng gì tới đầu ra.

Có đáng lo với MNIST không? Không. Ảnh MNIST 28×28 nhưng chữ số thật nằm gọn trong vùng 20×20 ở giữa, viền ngoài luôn là nền đen. Sau 2 lần pool, chỉ số 6 ở tầng 7×7 ứng với rìa ngoài cùng

ảnh gốc — gần như chắc chắn là nền.

Nhưng nó **bất đối xứng**: luôn cắt dưới/phải, không bao giờ cắt trên/trái. Muốn giữ thì dùng `ceil_mode=True` → ra 4×4 thay vì 3×3, và `fc` tăng từ 1,450 lên 2,570 params.

3. Mã giả

```
maxpool2d(input[CH][IN_H][IN_W],
          output[CH][OUT_H][OUT_W]):

    for c = 0 .. CH-1:                # từng kênh độc lập, không trộn kênh
        for oy = 0 .. OUT_H-1:
            for ox = 0 .. OUT_W-1:

                m = -INFINITY          # KHÔNG khởi tạo bằng 0

                for ky = 0 .. K-1:
                    for kx = 0 .. K-1:
                        iy = oy * STRIDE + ky    # không có PAD -> không dấu trừ
                        ix = ox * STRIDE + kx
                        if input[c][iy][ix] > m:
                            m = input[c][iy][ix]

                output[c][oy][ox] = m
```

5 vòng lặp, ít hơn `Conv2d` một vòng — vì không có `in_ch`, mỗi kênh xử lý riêng.

Không cần kiểm tra biên: `OUT_H` đã tính bằng `floor` nên mọi `iy`, `ix` sinh ra đều nằm trong ảnh. Phần bị `floor` cắt đơn giản là không bao giờ được chạm tới.

4. Áp dụng vào `SmallCNN`

Bước	H_in	Phép tính	H_out
pool sau conv1	28	$\text{floor}((28-2)/2)+1$	14
pool sau conv2	14	$\text{floor}((14-2)/2)+1$	7
pool sau conv3	7	$\text{floor}((7-2)/2)+1$	3 ← mất hàng/cột cuối

Chuỗi shape đầy đủ:

```
1×28×28 --conv1--> 8×28×28 --pool--> 8×14×14
          --conv2--> 16×14×14 --pool--> 16× 7× 7
          --conv3--> 16× 7× 7 --pool--> 16× 3× 3
          --flatten-> 144      --fc----> 10
```

$144 = 16 \times 3 \times 3$.

So với `DigitCNN` ($32 \times 7 \times 7 = 1568$), giảm 10.9 lần nhờ **hai đòn bẩy**: bớt channel 32→16 (2×) và thêm conv3+pool3 để 49→9 ô (5.4×).

5. Mức FPGA-friendly

✅ **Rất tốt.** Chỉ cần comparator, không nhân không chia. Rẻ hơn cả `Conv2d`.

May mắn là kiến trúc chọn **max** chứ không phải average. Average $2 \times 2 =$ chia 4 = dịch phải 2 bit, vẫn ổn; nhưng kernel không phải lũy thừa 2 thì phải chia thật — đắt trên phần cứng.

6. Chú ý khi port sang C

Bẫy	Chi tiết
Khởi tạo m	Dùng <code>-INFINITY</code> , không dùng 0. Trong model này pool đứng sau ReLU nên mọi giá trị ≥ 0 và khởi tạo 0 vẫn đúng — nhưng đối thứ tự lớp một lần là sai ngay. Viết đúng từ đầu
floor hay ceil	Phải khớp PyTorch. C chia số nguyên tự làm tròn xuống, nên $(n-k)/s + 1$ là đúng. Viết <code>ceil</code> thì shape lệch từ đây, mọi lớp sau sai theo
Không trộn kênh	Mỗi kênh pool độc lập. Lồng nhầm vòng <code>c</code> vào trong là sai về bản chất

7. Tự kiểm tra

- `MaxPool2d(2, 2)` trên ảnh 13×13 ra bao nhiêu? Bao nhiêu hàng/cột bị vớt?
- Trong SmallCNN, tổng cộng bao nhiêu phép **so sánh** ở cả 3 lần pool? (gợi ý: mỗi ô 2×2 cần 3 phép so sánh)
- Nếu đổi cả 3 pool sang `ceil_mode=True`, chuỗi shape thành gì? `flatten` ra bao nhiêu? `fc` bao nhiêu params? Tổng model bao nhiêu?

Đáp án

Câu 1 — `MaxPool2d(2, 2)` trên ảnh 13×13

```
H_out = floor((H_in - K) / STRIDE) + 1
        = floor((13 - 2) / 2) + 1
        = floor(5,5) + 1
        = 5 + 1 = 6
```

$13 \times 13 \rightarrow 6 \times 6$.

Bao nhiêu bị vớt: cửa sổ cuối bắt đầu ở chỉ số $(H_{out}-1) \times STRIDE = 10$, phủ tới chỉ số 11. Ảnh có chỉ số `0..12`, nên **chỉ số 12 — hàng cuối và cột cuối — không cửa sổ nào chạm tới, bị bỏ hoàn toàn**. Mất $13^2 - 12^2 = 25$ pixel.

Câu 2 — Tổng số phép so sánh ở cả 3 lần pool

Trước hết, **vì sao 3 phép chứ không phải 4** cho một ô 2×2 :

Cách viết	Số phép so sánh	Nhận xét
<code>m = -∞</code> rồi so cả 4 số	4	Phép đầu <code>a > -∞</code> luôn đúng , không mang thông tin
<code>m = a</code> rồi so 3 số còn lại	3	Phần tử đầu gán thẳng, không cần so

Mã giả gốc chọn cách `-∞` vì viết gọn (4 vòng lặp giống hệt nhau, không phải tách riêng phần tử đầu). Khi port sang C hoặc RTL mà muốn tiết kiệm thì tách phần tử đầu ra.

```
số so sánh 1 lớp pool = CH × OUT_H × OUT_W × 3

pool1:  8 × 14 × 14 × 3 = 4.704
pool2: 16 ×  7 ×  7 × 3 = 2.352
pool3: 16 ×  3 ×  3 × 3 =   432
        tổng = 7.488
```

Câu 3 — Đổi cả 3 pool sang `ceil_mode=True`

Công thức chỉ đổi `floor` thành `ceil`:

```
H_out = ceil((H_in - K) / STRIDE) + 1
```

Lớp pool	Vào	<code>floor</code> (hiện tại)	<code>ceil</code>	Đổi không?
pool1	28	<code>floor(13)+1 = 14</code>	<code>ceil(13)+1 = 14</code>	Không — 26/2 chia hết
pool2	14	<code>floor(6)+1 = 7</code>	<code>ceil(6)+1 = 7</code>	Không — 12/2 chia hết
pool3	7	<code>floor(2,5)+1 = 3</code>	<code>ceil(2,5)+1 = **4**</code>	Có

Chỉ `pool3` đổi, vì chỉ nó có `(H_in - K)` lẻ. Chuỗi shape mới:

```
1×28×28 → conv1 → 8×28×28 → pool → 8×14×14
        → conv2 → 16×14×14 → pool → 16×7×7
        → conv3 → 16×7×7   → pool → 16×4×4   ← chỗ duy nhất khác
```

	Hiện tại	<code>ceil_mode=True</code>
Shape trước <code>flatten</code>	16×3×3	16×4×4
<code>flatten</code>	144	256
<code>fc params</code>	1.450	2.570
Tổng model	5.018	6.138

Chỉ `fc` đổi — `params` của `conv` không phụ thuộc kích thước ảnh. Tổng: `5.018 - 1.450 + 2.570 = 6.138`.

Đánh đổi: giữ được hàng/cột cuối (không vớt thông tin), nhưng tốn thêm **1.120 tham số**, tức tăng 22%. Với MNIST thì chữ số nằm gọn giữa ảnh nên hàng/cột biên gần như trống — không đáng. Ngoài ra `ceil_mode` buộc cửa sổ cuối thò ra ngoài ảnh, trong C phải xử lý vùng thiếu đó, thêm một nhánh điều kiện nữa.